MÓDULO PROFESIONAL: PROGRAMACIÓN (CÓDIGO 0485)

Unidad 4: Colecciones de datos

Bloque I - Aplicación de las estructuras de almacenamiento (Arrays)

PROFESORADO Y CONTACTO
Profesor: Jorge Cabranes Azcona | Email: jca920@educa.madrid.org

UNIDAD 4: APLICACIÓN DE LAS ESTRUCTURAS DE ALMACENAMIENTO

CONTENIDOS

- Introducción
- Arrays unidimensionales
- Arrays bidimensionales
- Arrays irregulares y multidimensionales
- Copia y clonado de Arrays
- Ordenación de Arrays

Introducción

Un **array** (o **arreglo**) es una **estructura de datos de tamaño fijo** que permite **almacenar un conjunto de valores del mismo tipo** (por ejemplo, enteros, cadenas, objetos, etc.). En otras palabras, un array es como una **fila de casillas numeradas**, donde cada casilla contiene un valor.

Ejemplo conceptual:

casillas	[0]	[1]	[2]	[3]	[4]
valores	3	2	5	1	8

- En Java, **los índices comienzan en 0**, por lo que:
  - ✓ El primer elemento es `array[0]`
  - ✓ El último elemento es `array[array.length - 1]`

Introducción

- El tamaño de un array **no puede cambiar** después de crearse. Existen estructuras dinámicas, como ArrayList, HashMap, etc, que se verán más adelante.

- Cuando se crea un array, **sus posiciones se inicializan automáticamente con valores por defecto según el tipo**: byte, short, int, long =0 / float, double=0.0 / boolean=false / char= '\u0000' / Object, String=null.

- Todos los arrays tienen una **propiedad pública** llamada **length**, que indica el **número de elementos** que pueden almacenar.

```
int[] numeros = new int[5];
System.out.println(numeros.length);   // Muestra 5
```

Arrays Unidimensionales

Un **array unidimensional** (también llamado **vector**) se puede imaginar como una sola fila de datos.

Declaración de un array unidimensional

```
Tipo[] nombre = new Tipo[tamaño];
```

O alternativamente,

```
Tipo nombre []= new Tipo[tamaño];
```

Ejemplo:

```
int[] notas = new int[6];
int notas[] = new int[6]; // también válido
```

Array	Valores
notas[0]	0
notas[1]	0
notas[2]	0
notas[3]	0
notas[4]	0
notas[5]	0

Arrays Unidimensionales

Inicialización de un array unidimensional

- **Alternativa 1:** Declarar y asignar los valores uno a uno.

```
int[] notas = new int[6];
notas[0] = 5;
notas[1] = 3;
notas[2] = 1;
notas[3] = 8;
```

- **Alternativa 2:** Declarar en línea con inicialización directa de todos los valores entre llaves.

```
int[] notas = {5, 3, 1, 8};
```

Arrays Unidimensionales

Inicialización de un array unidimensional

- **Alternativa 3:** Usar estructuras repetitivas para recorrer el array e ir asignando el valor a sus elementos.

```
int[] notas = new int[6];
      for (int i = 0; i < notas.length; i++) {
         notas[i] = i * 2;
      }
// Resultado: [0, 2, 4, 6, 8, 10]
```

Entrada por teclado:

```
String[] paises = new String[8];
for (int i = 0; i < paises.length; i++) {
   paises[i] = JOptionPane.showInputDialog("Introduce país " + (i + 1));
}
```

Arrays Unidimensionales

Recorrido de un array unidimensional

- **Con bucle for clásico:**

```
int[] notas = {6, 7, 8, 1, 10, 5};
for (int i = 0; i < notas.length; i++) {
   System.out.print(notas[i] + " ");
}
```

- **Con bucle foreach (más simple):**

```
int[] notas = {1, 4, 5, 6, 7, 8, 3, 8};
for (int n : notas) {
   System.out.print(n + " ");
}
```

- **Mostrar el array completo fácilmente:**

```
System.out.println(Arrays.toString(notas)); // Salida: [1, 4, 5, 6, 7, 8, 3, 8]
```

Arrays Bidimensionales

Un **array bidimensional** (o **matriz**) se puede imaginar como una tabla con **filas y columnas**:

Ejemplo conceptual:

	Columnas		
Filas	**0**	**1**	**2**
0	[0][0] = 1	[0][1] = 4	[0][2] = 5
1	[1][0] = 5	[1][1] = 7	[1][2] = 2
2	[2][0] = 8	[2][1] = 3	[2][2] = 8

Declaración de un array bidimensional

```
Tipo[][] nombre = new Tipo[numFilas][numColumnas];
```

Ejemplo:

```
int[][] matriz = new int[3][3];
```

Arrays Bidimensionales

Inicialización de un array bidimensional

Un **array bidimensional** puede inicializarse de varias formas, igual que unidimensional, pero ahora necesitamos indicar **filas** y **columnas**.

- **Alternativa 1: Declaración e inicialización posterior con asignación valor a valor.** Primero declaramos el array y luego asignamos cada elemento usando **dos índices**: uno para la fila y otro para la columna.

```
int[][] matriz = new int[3][3];

    matriz[0][0] = 1;     matriz[1][0] = 4;     matriz[2][0] = 7;
    matriz[0][1] = 2;     matriz[1][1] = 5;     matriz[2][1] = 8;
    matriz[0][2] = 3;     matriz[1][2] = 6;     matriz[2][2] = 9;
```

Visualización

```
1  2  3

4  5  6

7  8  9
```

Arrays Bidimensionales

Inicialización de un array bidimensional

- **Alternativa 2: Inicialización en línea con llaves {}**
  Podemos declarar e inicializar el array en una sola línea utilizando **bloques de llaves anidados.**

```
int[][] matriz = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
};
```

Visualización:

```
1  2  3
4  5  6
7  8  9
```

Esta es la forma **más habitual** cuando conocemos los valores de antemano.

Arrays Bidimensionales

Inicialización de un array bidimensional

- **Alternativa 3: Inicialización mediante bucles**

  Podemos usar **bucles for anidados** para rellenar el array automáticamente. Un bucle anidado utilizará dos índices, cada índice se utilizará para una dimensión diferente. En Arrays bidimensionales, la condición del bucle externo debe hacer referencia a la cantidad de filas del Array. Mientras que la condición del bucle interno debe hacer referencia a la cantidad de elementos de la fila actual.

```
int[][] matriz = new int[3][3];
for (int fila = 0; fila < matriz.length; fila++) {
    for (int col = 0; col < matriz[fila].length; col++) {
        matriz[fila][col] = fila + col;
    }
}
```

Resultado:

```
0  1  2
1  2  3
2  3  4
```

Arrays Bidimensionales

Recorrido de array bidimensional

- **Con bucles anidados:**

```
for (int fila = 0; fila < matriz.length; fila++) {
    for (int col = 0; col < matriz[fila].length; col++) {
        System.out.print(matriz[fila][col] + " ");
    }
    System.out.println();
}
```

- **Con foreach anidado (más simple):**

```
for (int[] fila : matriz) {
    for (int valor : fila) {
        System.out.print(valor + " ");
    }
    System.out.println();
}
```

Arrays Bidimensionales

Recorrido de array bidimensional

- **Imprimir cada fila como texto:**

```
for (int[] fila : matriz) {
    System.out.println(Arrays.toString(fila));
}
```

Arrays bidimensionales irregulares

En Java, una **matriz irregular** es un array bidimensional en el que **cada fila puede tener una longitud distinta**, es decir, un número diferente de columnas. Es decir, todas las filas pertenecen a la misma matriz, pero no tienen el mismo número de columnas.

Declaración de un array bidimensional irregular

```
Tipo[][] nombre = new Tipo[numFilas][];
```

Fíjate que en la declaración **solo se indica el número de filas**. Las columnas se definirán después **fila a fila**, ya que cada una puede tener un tamaño distinto.

Arrays bidimensionales irregulares

Inicialización de un array bidimensional irregular

- **Alternativa 1: Declaración e inicialización por partes**
  Primero se crea el array con las filas, y luego se asigna a cada fila un número distinto de columnas.

```
int[][] matriz = new int[3][];   // 3 filas, sin columnas definidas

matriz[0] = new int[2];       // 2 columnas en la fila 0
matriz[1] = new int[4];       // 4 columnas en la fila 1
matriz[2] = new int[3];       // 3 columnas en la fila 2

// Asignamos valores
    matriz[0][0] = 10; matriz[0][1] = 20;
    matriz[1][0] = 30; matriz[1][1] = 40; matriz[1][2] = 50; matriz[1][3] = 60;
    matriz[2][0] = 70; matriz[2][1] = 80; matriz[2][2] = 90;
```

Representación visual:

```
Fila 0:[10, 20]
Fila 1:[30, 40, 50, 60]
Fila 2:[70, 80, 90]
```

Arrays bidimensionales irregulares

Inicialización de un array bidimensional irregular

- **Alternativa 2: Declaración en línea con llaves {}**
  Podemos crear directamente la matriz irregular especificando las filas y columnas con distintos tamaños:

```
int[][] irregular = {
        {1, 2, 3},       // 3 columnas
        {4, 5, 6, 7},    // 4 columnas
        {8, 9}           // 2 columnas
};
```

Representación visual:

```
[1, 2, 3]
[4, 5, 6, 7]
[8, 9]
```

En este caso:

```
irregular[0].length = 3
irregular[1].length = 4
irregular[2].length = 2
```

Arrays bidimensionales irregulares

Inicialización de un array bidimensional irregular

- **Alternativa 3: Inicialización mediante bucles**
  Podemos crear la matriz irregular dinámicamente, asignando tamaños de fila según una fórmula o valor calculado.

```
int[][] matriz = new int[3][];

// Cada fila tendrá un número de columnas igual a (fila + 2)
for (int i = 0; i < matriz.length; i++) {
      matriz[i] = new int[i + 2];
  }
// Asignamos valores con bucles anidados
for (int i = 0; i < matriz.length; i++) {
      for (int j = 0; j < matriz[i].length; j++) {
         matriz[i][j] = i + j;
      }
  }
```

Resultado visual:

```
[0, 1]
[1, 2, 3]
[2, 3, 4, 5]
```

Arrays bidimensionales irregulares

Recorrido de un array bidimensional irregular

El recorrido se hace igual que en una matriz regular, usando length tanto para filas como para columnas.

```
for (int i = 0; i < matriz.length; i++) {
      for (int j = 0; j < matriz[i].length; j++) {
         System.out.print(matriz[i][j] + " ");
      }
      System.out.println();
   }
```

O bien con **foreach anidados**:

```
for (int[] fila : matriz) {
      for (int valor : fila) {
         System.out.print(valor + " ");
      }
      System.out.println();
   }
```

Arrays multidimensionales

En Java, se pueden crear arrays con **más de dos dimensiones**, aunque **no son muy habituales en programación básica**.

Declaración de un array multidimensional

```
Tipo[][]...[] nombre = new Tipo[n1][n2]...[nk];
```

- Tipo → tipo de los elementos (int, double, String, etc.)
- n1, n2, ..., nk → tamaño de cada dimensión
- k → número de dimensiones

Para cada nueva dimensión añadimos un par de corchetes [].

Arrays multidimensionales

Inicialización de un array multidimensional

Un ejemplo de inicialización con llaves:

```
int[][][] cubo = {{{1, 2, 3}, {4, 5, 6}}, {{7, 8, 9}, {10, 11, 12}}};
```

Es un **array tridimensional** (int[][][]), es decir, un conjunto de **"planos"** (o matrices 2D) apilados en la tercera dimensión.

Dimensiones:

- 2 planos (cubo.length = 2)
- Cada plano tiene **2 filas**
- Cada fila tiene **3 columnas**

Por tanto, sus dimensiones son: 2 × 2 × 3

NOTA: Aunque en Java hablamos de arrays tridimensionales y a veces los llamamos 'cubos', solo son cubos en sentido estricto si todas las dimensiones son iguales. Lo importante es que hay tres niveles de índices."

Representación visual

Plano 0:

```
1  2  3
4  5  6
```

Plano 1:

```
7   8   9
10  11  12
```

Acceso a elementos

```
cubo[0][0][0]= 1
cubo[0][1][2] = 6
cubo[1][0][1] = 8
cubo[1][1][2] = 12
```

Arrays multidimensionales

Ejemplo de array tridimensional (3D)

```
int[][][] cubo = new int[3][3][3]; // 3D: filas, columnas y profundidad

// Asignamos valores de ejemplo
int valor = 1;
for (int i = 0; i < cubo.length; i++) {          // recorre filas
    for (int j = 0; j < cubo[i].length; j++) {   // recorre columnas
        for (int k = 0; k < cubo[i][j].length; k++) { // recorre profundidad
            cubo[i][j][k] = valor++;
        }
    }
}
```

Visualización conceptual del cubo:

Plano 0:	Plano 1:	Plano 2:
1 2 3	10 11 12	19 20 21
4 5 6	13 14 15	22 23 24
7 8 9	16 17 18	25 26 27

Arrays multidimensionales

Recorrido de un array multidimensional

Para **recorrer un array n-dimensional** usamos un **bucle por cada dimensión**, similar a como hicimos con arrays bidimensionales.

```
for (int i = 0; i < cubo.length; i++) {
      for (int j = 0; j < cubo[i].length; j++) {
         for (int k = 0; k < cubo[i][j].length; k++) {
            System.out.print(cubo[i][j][k] + " ");
         }
         System.out.println();
      }
      System.out.println("--- Fin del plano ---");
   }
```

Copia y clonado de Arrays

Copia por referencia en Arrays

En Java, las **variables no primitivas** (como los objetos y los arrays) **no almacenan directamente los datos**, sino una **referencia** (una dirección en memoria) al lugar donde está guardado el objeto. Esto significa que si hacemos una **asignación directa** entre dos arrays, ambas variables apuntarán al **mismo espacio de memoria**.

Por tanto, **si modificamos un elemento en una de ellas, el cambio se reflejará también en la otra.**

```
int[] numeros = {10, 20, 30};
int[] copiaReferencia = numeros; // copia por referencia

copia[0] = 99; // modificamos la copia

System.out.println(Arrays.toString(numeros));          // [99, 20, 30]
System.out.println(Arrays.toString(copiaReferencia));  // [99, 20, 30]
```

Copia y clonado de Arrays

Clonado de Arrays

En Java, los **arrays son objetos**, y todos los objetos heredan el método clone(), que crea una **copia exacta del contenido** en un nuevo espacio de memoria. Esto permite obtener un **array independiente del original** (una *copia profunda superficial*, ya que copia los elementos, no las referencias internas).

```
int[] primos = {1, 2, 3, 5, 7};
int[] copiaClonada = primos.clone();

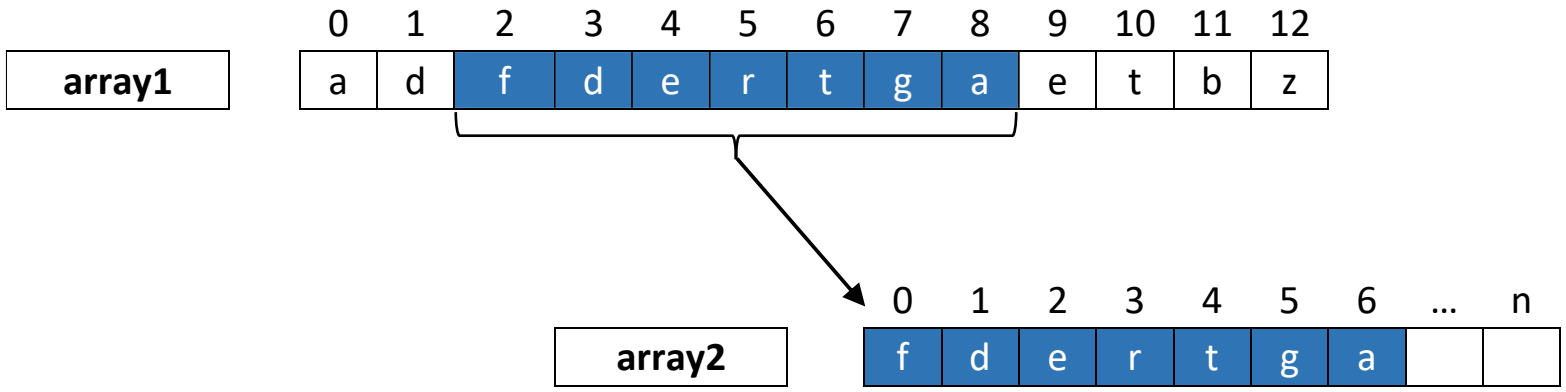
copiaClonada[0] = 0;

System.out.println(Arrays.toString(primos));      // [1, 2, 3, 5, 7]
System.out.println(Arrays.toString(copiaClonada)); // [0, 2, 3, 5, 7]
```

En este caso, el cambio en copiaClonada **no afecta al original**, ya que son dos arrays distintos en memoria.

Copia y clonado de Arrays

Copia parcial de Arrays con System.arraycopy()



Copia y clonado de Arrays

Copia parcial de Arrays con System.arraycopy()

Otra forma de copiar datos entre arrays es el método:

```
System.arraycopy(arrayOrigen, inicioOrigen,
arrayDestino, inicioDestino, cantidadElementos);
```

Parámetros:

- **arrayOrigen:** Array del que se copian los elementos.
- **inicioOrigen:** Posición inicial del array origen.
- **arrayDestino:** Array donde se copiarán los datos.
- **inicioDestino:** Posición inicial en el array destino.
- **cantidadElementos:** Número de elementos a copiar.

Copia y clonado de Arrays

Copia parcial de Arrays con System.arraycopy()

Ejemplo:

```
import java.util.Arrays;

   public class EjemploCopiaArray {
      public static void main(String[] args) {
         int[] primos = {1, 2, 3, 5, 7, 11, 13};
         int[] copia = new int[primos.length];

         // Copia 3 elementos desde la posición 1 de 'primos' a la posición 3 de 'copia'
         System.arraycopy(primos, 1, copia, 3, 3);

         System.out.println("Array original: " + Arrays.toString(primos));
         System.out.println("Array copia:    " + Arrays.toString(copia));
      }
   }
```

Salida:

```
Array original: [1, 2, 3, 5, 7, 11, 13]
Array copia:    [0, 0, 0, 2, 3, 5, 0]
```

Ordenación de Arrays

Ordenar los elementos de una colección (array, lista, etc.) permite:

- Buscar datos con mayor rapidez.
- Comparar y agrupar información de forma eficiente.
- Facilitar el tratamiento posterior de los datos.

Algoritmos de ordenación

Un **algoritmo de ordenación** define **cómo reorganizar los elementos** de una colección para que sigan un orden determinado (por ejemplo, de menor a mayor o de A a Z).

Podemos ordenar cualquier tipo de dato **ordenable**:

- Números (int, double, etc.)
- Cadenas de texto (String)
- Objetos (si implementan Comparable o usamos un Comparator)

Ordenación de Arrays

Existen muchos algoritmos de ordenación. Algunos son **simples pero lentos** (como el método burbuja) y otros **más complejos y eficientes** (como Quicksort o Mergesort).

El método de la burbuja

El **algoritmo de la burbuja (Bubble Sort)** es uno de los métodos más sencillos y conocidos. Este algoritmo compara elementos **adyacentes** y los **intercambia** si están desordenados. Tras cada pasada, el **mayor valor "flota"** hacia el final del array, como una burbuja.

Funcionamiento paso a paso

1. Compara el primer par de elementos ([0] y [1]).
2. Si están en orden incorrecto, los intercambia.
3. Continúa con el siguiente par ([1] y [2]), y así sucesivamente.
4. Tras la primera pasada, el mayor valor está al final.
5. Repite el proceso descartando el último elemento en cada iteración.

Ordenación de Arrays

El método de la burbuja: Ejemplo de implementación

```
    public static int[] burbuja(int[] vector) {
        int aux;
        for (int i = 0; i < vector.length; i++) {
            for (int j = 0; j < vector.length - i - 1; j++) {
                // Si el elemento actual es mayor que el siguiente, los intercambiamos
                if (vector[j] > vector[j + 1]) {
                    aux = vector[j];
                    vector[j] = vector[j + 1];
                    vector[j + 1] = aux;
                }
            }
        }
        return vector;
    }
    // Ejemplo de uso
    int[] arrayEjemplo = {3, 6, 9, 6, 1, 2, 3};
System.out.println("Sin ordenar: " + Arrays.toString(arrayEjemplo));

    int[] arrayOrdenado = burbuja(arrayEjemplo);
System.out.println("Ordenado: " + Arrays.toString(arrayOrdenado));
```

No recomendable para arrays grandes.

Ordenación de Arrays

Algoritmo de ordenación rápida (Quicksort)

El **Quicksort** es uno de los algoritmos más rápidos y eficientes en la práctica. Utiliza la estrategia **divide y vencerás** para ordenar grandes colecciones.

Funcionamiento paso a paso

1. Elegir un **pivote** (normalmente el primer elemento).
2. Colocar todos los elementos **menores** que el pivote a su izquierda y los **mayores** a su derecha.
3. El pivote queda en su **posición definitiva**.
4. Repetir el proceso **recursivamente** con las sublistas izquierda y derecha.

Ordenación de Arrays

Algoritmo de ordenación rápida (Quicksort)

Ejemplo de implementación

```
public static void quicksort(int[] A, int izq, int der) {
      int pivote = A[izq];  // Elegimos el primer elemento como pivote
      int i = izq;
      int j = der;
      int aux;
      while (i < j) {
         while (A[i] <= pivote && i < j) i++; // Busca elemento mayor que el pivote
         while (A[j] > pivote) j--;       // Busca elemento menor que el pivote
         if (i < j) { // Intercambio de elementos
            aux = A[i];
            A[i] = A[j];
            A[j] = aux;
         }
      }
```

Ordenación de Arrays

Algoritmo de ordenación rápida (Quicksort)

Ejemplo de implementación

```
        // Colocamos el pivote en su lugar definitivo
        A[izq] = A[j];
        A[j] = pivote;

        // Ordenamos las sublistas de forma recursiva
        if (izq < j - 1) quicksort(A, izq, j - 1);
        if (j + 1 < der) quicksort(A, j + 1, der);
    }

    // Ejemplo de uso
    int[] datos = {9, 3, 7, 2, 8, 5, 1, 4, 6};
    quicksort(datos, 0, datos.length - 1);
System.out.println(Arrays.toString(datos));
```

Ordenación de Arrays

Otros algoritmos de ordenación

Existen muchos más, como:

- **Mergesort** → muy estable, también O(n log n)
- **Heapsort** → eficiente y con buen rendimiento en el peor caso
- **Insertion Sort** → útil para arrays pequeños o casi ordenados

Ordenación de Arrays

Búsqueda binaria

La **búsqueda binaria** es un algoritmo que **localiza un elemento dentro de una colección ordenada**, reduciendo a la mitad el número de elementos que hay que comprobar en cada paso. Es mucho más rápida que la **búsqueda secuencial.**

Funcionamiento paso a paso

El algoritmo **divide el array en mitades** sucesivas hasta encontrar el valor buscado (o determinar que no está):

1. Se busca el **elemento central** del array.
2. Se compara el valor del **centro** con el valor **buscado**:
   - Si son **iguales**, hemos encontrado el elemento
   - Si el valor buscado es **menor**, se repite el proceso **en la mitad izquierda** del array.
   - Si el valor buscado es **mayor**, se repite **en la mitad derecha**.
3. El proceso se repite (recursiva o iterativamente) hasta encontrar el valor o quedarse sin elementos.

Ordenación de Arrays

Búsqueda binaria

Ejemplo visual

Imaginemos un array ordenado de notas: `[4.0, 5.0, 6.5, 7.5, 8.0, 9.0]`
Queremos buscar la nota 7.5.

Paso	Límite inferior	Límite superior	Elemento central	Comparación	Acción
1	0	5	índice 2 → 6.5	6.5 < 7.5	Buscar en la mitad derecha
2	3	5	índice 4 → 8.0	8.0 > 7.5	Buscar en la mitad izquierda
3	3	3	índice 3 → 7.5	7.5 = 7.5	Encontrado

Solo necesitó **3 comparaciones** en lugar de 6 (como haría una búsqueda secuencial).

Ordenación de Arrays

Búsqueda binaria

Ejemplo de implementación recursiva

```
public static boolean busquedaBinaria(int[] numeros, int buscado, int inicio, int fin) {
    if (inicio > fin)
        return false; // No encontrado

    int mitad = (inicio + fin) / 2;

    if (numeros[mitad] == buscado)
        return true;
    else if (numeros[mitad] < buscado)
        return busquedaBinaria(numeros, buscado, mitad + 1, fin); // Buscar en la mitad derecha
    else
        return busquedaBinaria(numeros, buscado, inicio, mitad - 1); // Buscar en la mitad izquierda
}
```

Ordenación de Arrays

Búsqueda binaria

Ejemplo de implementación recursiva

```
public static void mostrarBusqueda(int[] numeros, int buscado) {
    if (busquedaBinaria(numeros, buscado, 0, numeros.length - 1))
        System.out.println("El número " + buscado + " está en el array.");
    else
        System.out.println("El número " + buscado + " NO está en el array.");
}
```

El famoso String[] args en Java

En todos los programas Java aparece la siguiente cabecera principal:

```
public static void main(String[] args) {
```

Este es el **método principal** que se ejecuta al iniciar una aplicación, y su estructura tiene un significado concreto:

- **public** → el método puede ser accedido desde fuera de la clase.
- **static** → no es necesario crear un objeto para poder ejecutarlo.
- **void** → no devuelve ningún valor.
- **String[] args** → es un **array de cadenas (Strings)** que contiene los **argumentos** que se pasan al programa desde la consola.

Esto significa que cuando ejecutamos un programa Java con el comando java, podemos enviarle una lista de palabras separadas por espacios, que se almacenarán dentro del array args.

El famoso String[] args en Java

Ejemplo 1. Argumentos de texto

```
class HolaMundo {
   public static void main(String[] args) {
      System.out.print("Hola");
      for (int i = 0; i < args.length; i++) {
         System.out.print(" " + args[i]);
      }
      System.out.println(", ¿qué tal estás?");
   }
}
```

Ejecución por consola:

```
java HolaMundo
Hola, ¿qué tal estás?

java HolaMundo Pepe
Hola Pepe, ¿qué tal estás?

java HolaMundo Pepe García
Hola Pepe García, ¿qué tal estás?
```

Observa que todos los valores escritos tras el nombre de la clase se almacenan en el array args.

El famoso String[] args en Java

Ejemplo 2. Argumentos combinados (texto y número)

Podemos procesar los argumentos para convertirlos en números y usarlos en el programa.

```
class HolaMundo2 {
   public static void main(String[] args) {
      if (args.length > 1) {
         int repeticiones = Integer.parseInt(args[1]);
         for (int i = 0; i < repeticiones; i++) {
            System.out.println("Hola " + args[0]);
         }
      } else {
         System.out.println("Hola mundo simple");
      }
   }
```

Ejecución por consola:

```
java HolaMundo2
Hola mundo simple

java HolaMundo2 Pepe
Hola mundo simple

java HolaMundo2 Pepe 3
Hola Pepe
Hola Pepe
Hola Pepe

java HolaMundo2 Pepe García
Error (ya que "García" no puede
convertirse en número)
```

El famoso String[] args en Java

Importante

- Los **argumentos de consola** siempre se reciben como **cadenas de texto**.
  Si necesitamos números, debemos convertirlos usando métodos como Integer.parseInt() o Double.parseDouble().
- El **número de argumentos** se obtiene con args.length.